

암기가 아니라 적용을 배운다: 정책-인-컨텍스트 파인튜닝은 처음 보는 조직 정책에 일반화된다

초고 v0.6 한국어판 (2026-09-01) — 영어판 DRAFT.md 와 병행 유지되는 번역본입니다(제출 정보는 영어판). 모든 수치는 이 저장소에서 실측·재현 가능합니다(\$7). **body** 표시는 원문 본문 확인이 남은 주장이었으나, TriMPI·PRT 2건은 본문 확인 까지 완료되었습니다.

초록

회사에서 업무를 맡은 담당자는 대개 프로그램을 짜거나 모델을 고르는 전문가가 아닙니다. 그리고 그 업무를 규정하는 문서 — 정책, 런북, 체크리스트 — 는 조직의 사정에 따라 수시로 바뀝니다. 우리의 출발점은 단순한 작업 흐름입니다. 담당자가 먼저 에이전트와 대화하며 일을 해내면, 그 기록을 선언적 작업 언어 OpenWorkLang으로 표현하고 OpenWorkCompiler로 컴파일합니다. 그러면 정책은 온톨로지와 규칙이 되고, 계산은 프로그램이 되고, 판단은 로컬 소형 모델이 맡습니다.

이 논문이 실험으로 확인하는 것은 이 사슬에서 가장 의심스러운 고리입니다. 소형 모델이 특정 정책을 암기하는 게 아니라, 컨텍스트로 주어진 어떤 정책이든 적용하는 판단의 기술을 배울 수 있는가? 결정론 규칙 엔진이 라벨한 조직 결정 34개 사례로 훈련한 7B 모델은 **훈련 때 본 적 없는 정책에서도 판단 정확도가 크게 올랐습니다**. 미훈련 모델 대비 다섯 배 가까운 개선이며, 정책 하나하나를 번갈아 홀드아웃하는 leave-case-out으로 재 확인했습니다. 개선의 정체를 통제 실험으로 좁혔습니다. 상식과 반대로 뒤집은 정책에서도 모델은 뒤집힌 대로 판단했고 — 사전지식으로 답하면 0점이 되는 조건입니다 — 예시만 몇 개 주는 방식보다 훈련이 뚜렷하게 앞섰습니다. 훈련 시드를 바꿔도 개선은 유지됐습니다.

경계도 같이 드러났습니다. 똑같은 레시피를 바이트 단위로 정확해야 하는 산술에 적용하면 통과율이 바닥이었고, 계산 과정을 학습 데이터에 넣어도, 도구 없는 프런티어 모델을 써도 마찬가지였습니다. 그래서 계산은 실행할 때마다 LLM에 맡기지 않고 한 번 코드로 컴파일합니다. 컴파일의 출발점이 조직이 이미 지불한 작업 세션이라, 두 번째 실행부터는 frontier 토큰이 남습니다. 세부 수치와 신뢰구간은 \$5에 있습니다.

1. 서론

조직에서 무엇이 올바른 일 처리인지는 문서가 정합니다. 결정 정책, 지원 런북, 에스컬레이션 매트릭스, 채점 루브릭, 편집 스타일 가이드, 컴플라이언스 체크리스트가 다 그런 문서입니다. 그리고 이 문서들은 모델이 아니라 조직의 사정에 따라 수시로 바뀝니다. 따라서 문서 위에서 AI를 운영하려면 두 가지를 분리해야 합니다. 오늘의 정책 내용은 바뀌는 즉시 교체할 수 있도록 프롬프트 컨텍스트에 두고, 그 정책을 읽어 개별 사안에 적용하는 **기술**은 모델이 학습하게 하는 방식입니다. 이 논문은 프런티어 모델이 성공적으로 처리한 기록을 학습한 소형 모델이, 훈련에 없던 문서에도 이 기술을 적용할 수 있는지를 측정합니다.

이 분해는 실무에서 이렇게 굴러갑니다. 회사의 담당자는 프로그램을 짜거나 모델을 고르는 전문가가 아닙니다. 처음에는 자기 직군의 일 — 견적, 환불 승인, 장애 분류 — 을 에이전트와 대화하며 그냥 해냅니다. 그런데 그 대화 기록에는 그 일의 정책과 계산법과 판단 방식이 이미 들어 있습니다. 이를 선언적 작업 언어(OpenWorkLang)로 표현하고 컴파일러(OpenWorkCompiler)를 돌리면 정책은 온톨로지와 규칙이 되고, 계산은 프로그램이 되고, 판단은 훈련된 소형 모델이 맡습니다. 이렇게 하면 프런티어 모델을 매번 호출하지 않고도 로컬 GPU에서 특수 목적 업무를 처리할 수 있습니다. 이 논문이 확인하는 핵심은 그 과정에서 소형 모델이 실제로 정책 판단 능력을 익히는지입니다. 계산은 프로그램으로 컴파일해야 한다는 경계와 반복 실행의 비용 효과도 각각 \$5.3과 \$5.4에서 함께 검증합니다.

이 부류에서 결정론 채점이 되는 가장 깨끗한 사례가 조직의 결정 정책이라, 우리는 그것을 연구합니다. 직원이 "이 고객에게 12% 할인을 줘도 되나요?"라고 물을 때 조직이 원하는 답은 어느 똑똑한 모델의 의견이 아닙니다. **조직 자신의 정책이 적용된 결과**입니다. 순서 있는 규칙을 위에서부터 검사하고, 처음 걸리는 규칙을 인용하고, 정책이 지정한 승인자에

게 넘기고, 정책이 일부러 열어둔 재량 밴드("5-10%: AI 추천 가능, 팀장 승인")만 추천으로 채우는 것. 여기서 ML 문제의 조건 세 가지가 나오는데, 셋 다 위의 분해를 그대로 보여줍니다:

1. **문서는 모델보다 빨리 바뀐다.** 정책 P를 암기한 모델은 P가 P'이 되는 날 틀린다. 그러니 내용은 가중치 바깥에 두어야 한다.
2. **판단은 감사 가능해야 한다.** "승인"만으로는 답이 아니다. "승인, 본부장 라우팅, strategic_retention_band 인용, 그 조건이 이 레코드에 실제로 성립"까지가 답이다.
3. **재량은 선언되는 것이지 추론되는 것이 아니다.** 모델이 어디까지 추천해도 되는지는 문서가 정한다. 신뢰도 임계값이 정하는 게 아니다.

선행 연구는 셋 중 하나를 합니다. 정책을 가중치에 내재화하거나(Multimodal Policy Internalization / TriMPI, arXiv:2510.09474), 정책 지침을 준 에이전트를 벤치마킹하거나(τ -bench, 2406.12045; RuleArena, 2412.08972), 규칙이 개정될 때 얼마나 버티는지 측정합니다(GuideBench, 2505.11368). 정작 배포 현장이 요구하는 조합은 빠져 있습니다. 로컬에서 돌릴 수 있는 소형 모델에 "어떤 정책이 모든 컨텍스트에서 읽고 적용하는 기술"을 파인튜닝으로 가르치고, 훈련 때 본 적 없는 정책에서 그 기술을 재는 것 말입니다.

선언적 결정 정책을 고른 이유는 둘입니다. 적용 기술(순서대로 검사, 첫 일치 적용, 근거 인용)이 만만치 않으면서도, 정답은 바이트 단위로 채점할 수 있기 때문입니다. 런북이나 루브릭처럼 더 지저분한 문서로도 이 레시피가 통하는지는 아직 열린 질문이고, §6에서 로드맵과 함께 범위를 긋습니다.

기여:

- **과제와 코퍼스 (§3):** 10개 조직 기능의 결정 사례 34개를 순서 규칙과 명시적 재량 범위로 선언하고, 결정론 엔진으로 근거 규칙과 성립 조건까지 갖춘 학습·평가용 판단을 생성했습니다.
- **엄격한 결정론 채점기 (§3.3):** 판정·라우팅·파라미터·인용 규칙의 정확 일치 + 인용 규칙의 조건이 레코드에 실제로 성립하는지 검증(판정을 맞혀도 근거 날조는 실패).
- **새 정책으로의 전이 (§5.1):** 모든 정책을 차례로 학습에서 제외해 평가했을 때, 튜닝 모델은 미튜닝 모델과 인컨텍스트 학습 기준선을 꾸준히 앞섰습니다.
- **반사실 통제 (§5.2):** 상식이나 원래 정책을 따르면 반드시 틀리도록 뒤집은 정책에서도 성능이 유지돼, 모델이 세상의 기본값이 아니라 정책을 읽는 법을 익혔음을 확인했습니다.
- **적용 범위와 배치 경제학 (§5.3-5.4):** 같은 방법은 바이트 단위로 정확해야 하는 산술에는 효과가 없었고, 도구 없는 프런티어 모델도 같은 한계를 보였습니다. 따라서 계산은 코드로 컴파일하고 정책 판단만 훈련된 소형 모델에 맡기는 배치 원칙을 제시합니다.

2. 관련 연구

확장 예정; 조사로 검증된 위치 설정:

정책 내재화 vs 정책-인-컨텍스트. Multimodal Policy Internalization(TriMPI, 2510.09474)은 정책을 모델 파라미터 안으로 훈련해 추론 시 정책이 컨텍스트에 없어도 되게 합니다 — 우리와 정확히 반대의 유지보수 베틱입니다. 그쪽의 "Policy Override" 설정(그 논문 §5.2)은 갱신된 정책을 추론 시 컨텍스트로 주되 이를 내재화 모델의 *일반화 프로브*로 씁니다; 우리는 컨텍스트 적용 자체를 훈련 목표로 만듭니다. Policy Reasoning Traces(2509.23291)는 HIPAA·GDPR 등 특정 정책의 컴플라이언스 평가를 정책별 인스턴스 분할로 개선하며, 교차-정책 전이는 부수 분석(§4.2/App. G)으로만 등장합니다 — 우리의 홀드아웃-정책 프로토콜과 다릅니다. GuideBench(2505.11368)는 LLM 에이전트의 규칙 개정 강건성을 벤치마킹합니다.

정책 준수 벤치마크. τ -bench(2406.12045)는 도메인 정책 지침을 준 SOTA 에이전트를 평가하고, RuleArena(2412.08972)의 발견은 규칙 식별 실패와 규칙이 요구하는 계산의 실패를 구분해 우리 §5.3의 경계를 직접

예고하며, CRMArena-Pro(2505.18878)는 기밀 인지 등 엔터프라이즈 시나리오에서 에이전트를 평가합니다. 동시대의 엔터프라이즈 벤치마크(2605.08761, 초록 검증됨)는 규칙 엔진이 채점하는 결정 벤치마크를 독립적으로 구축했으나 평가 전용이고 재량 밴드가 없습니다. DMN(OMG 표준)은 FIRST-hit 의미론의 순서 결정 규칙을 10년째 표준화해 왔습니다 — 우리 DSL은 의도적으로 그 계열이며, ML 기여는 규칙 언어가 아니라 전이 측정입니다.

SFT와 산술. Faith and Fate(2305.18654)는 Transformer가 합성적 다단계 산술에 실패하고 추론을 선형화된 패턴 매칭으로 환원함을 보였고, "SFT Memorizes, RL Generalizes"(2501.17161)는 SFT가 암기하며 분포 밖에서 고전함을 보였으며, PAL(2211.10435)은 풀이 단계를 파이썬 인터프리터로 넘깁니다 — 계산을 코드로 라우팅하는 계열의 원조. 우리의 기여는 부정 결과 자체가 아니라 그 **짜지움**입니다: 산술에서 실패한 그 레시피·코퍼스 규모·게이트가 규칙 적용에서는 성공하고, 도구 없는 프런티어가 같은 산술 게이트에 실패해 "소형 모델은 계산을 못 한다"를 "추론은 계산을 못 한다"로 버립니다.

소형 모델 배포. Belcak et al.의 SLM 포지션 페이퍼(2506.02153)는 소형 모델이 에이전틱 AI의 미래라 주장하고 LLM→SLM 전환 알고리즘을 제시합니다; 우리는 고가치 슬롯 하나(정책 적용)의 훈련된 기술 레시피와 전이 측정, 그리고 다른 슬롯(파생)에 대해 프런티어조차 실패하는 결정론 게이트를 제공해 선호가 아닌 **배치 규칙**을 내놓습니다.

3. 과제와 코퍼스

3.1 결정 사례

*사례*는 반복되는 조직 결정 하나를 선언합니다: 결정 역할, **온톨로지**(개체·속성·관계), 피처 공간, **순서 규칙**(when 조건 → 판정·라우팅·파라미터·사유), 선택적 **재량 밴드**(defer: slm_recommend, 밴드 한계는 파라미터로), 그리고 fallback. 예시(축약):

```
id: sales-discount-approval # org: 영업 · role: 영업대표
rules:
  - {name: min_margin_violation, when: margin_pct - requested_discount_pct < 10, verdict: reject}
  - {name: strategic_retention_band,
    when: segment == strategic and churn_risk == high and requested_discount_pct <= 12,
    verdict: approve, route: 본부장, set: {max_discount_pct: 12}}
  - {name: over_director_threshold, when: requested_discount_pct > 10, verdict: escalate, route: 본부장}
  - {name: standard_auto, when: requested_discount_pct <= 5, verdict: approve, route: auto}
  - {name: gray_band_recommend, when: requested_discount_pct <= 10,
    defer: slm_recommend, route: 팀장, set: {recommend_between: [5, 10]}}
fallback: {verdict: escalate, route: 본부장}
```

코퍼스는 10개 기능(영업·재무·고객지원·인사·구매·법무·IT운영·마케팅·물류·보안)의 34개 사례를 담습니다. 결정론 엔진이 각 사례의 피처 공간에서 레코드를 샘플링하고 첫 일치 의미론을 적용해 **사례당 라벨된 판단 100건**(총 3,400건)을 만들며, 각 판단에는 적용 규칙명·조건·사유가 붙습니다. 생성은 시드 고정·바이트 재현입니다.

3.2 정책-인-컨텍스트 형식

모델에는 매 사례마다 조직·역할·결정을 설명하는 시스템 메시지, 해당 사례의 온톨로지와 **전체 순서 규칙 목록**(YAML 원문), 그리고 판단할 레코드를 제공합니다. 모델은 {verdict, route, params, cited_rule, rationale} 형식의 JSON 하나를 반환합니다. 특정 정책의 내용을 가중치에 저장하지 않으므로, 정책을 고치면 재훈련 없이 다음 판단부터 바뀐 규칙을 적용할 수 있습니다.

3.3 채점

이 과제에서 정답은 단순히 승인·거절을 맞히는 것이 아닙니다. **정확(exact)** 판정을 받으려면 판정, 라우팅, 파라미터가 엔진의 결정과 같아야 하며, 인용한 규칙도 첫 번째로 일치한 규칙이어야 합니다. 또 그 규칙의 조건이 실제 레코드에서 성립해야 합니다. 이 마지막 검사는 결과만 맞힌 뒤 근거를 꾸며내는 답을 걸러냅니다. **완화(relaxed)** 지표는 나머지가 모두 맞았을 때, 첫 일치 규칙은 아니더라도 실제로 성립하는 규칙을 인용한 답을 인정합니다. 첫 일치는 운영상 규칙이지 의미 자체의 필수 조건은 아니기 때문입니다. 판정만의 정확도도 함께 보고하며, 신뢰구간은 정책별 상관관계를 반영한 **사례-클러스터 부트스트랩**(사례 복원추출 10,000회)으로 계산합니다.

3.4 정직한 범위

이 코퍼스는 합성 데이터이며 라벨을 만든 엔진이 채점도 합니다. 정책은 이미 기계가 읽을 수 있는 형식이고, 규칙 조건의 필드명도 레코드 키와 일치합니다. 따라서 산문 정책에서 생기는 엔티티 정합 문제, 결측·노이즈 필드, 주식자 간 불일치는 포함하지 않습니다. 이 논문의 주장은 의도적으로 정제된 환경에서 측정한 **선언적 정책 적용**의 전이로 한정하며, 산문 정책으로 확장하는 방법은 §6에서 다룹니다.

4. 실험 설정

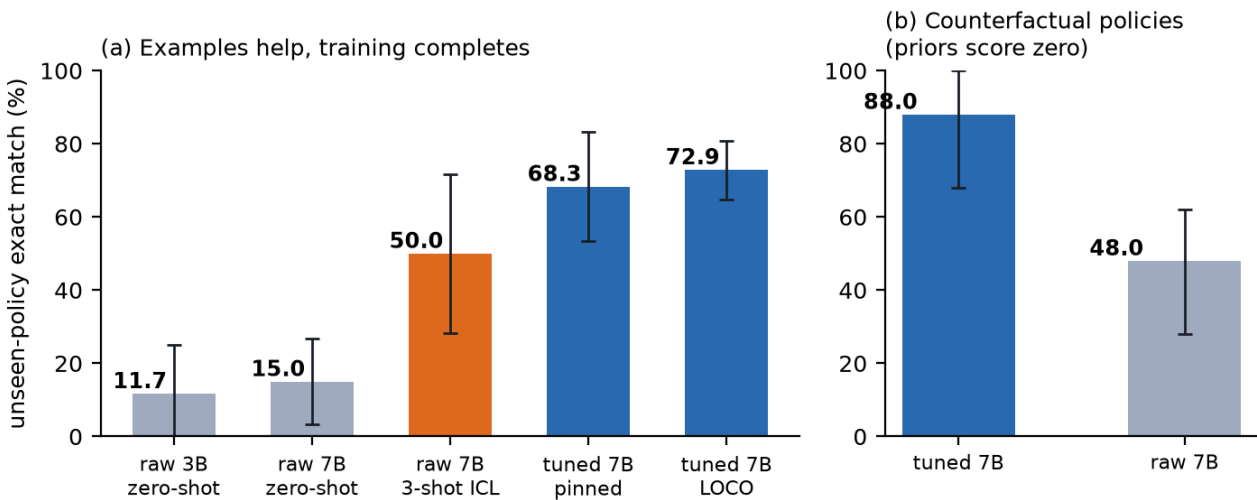
모델·훈련. Qwen2.5-7B-Instruct, QLoRA($r=16$, $\alpha=32$, dropout 0.05, 전체 선형 프로젝션), completion-only 손실, 2 에폭, LR $1e-4$ cosine, RTX 4090 1장에서 bf16 연산(런당 ≈ 12 분). 훈련 데이터: 28사례 $\times$ 40인스턴스(1,120행); 6사례는 **통째로** 홀드아웃. 환경은 핀 고정(requirements-train.txt) 이고 모든 런이 매니페스트(시드·에폭·데이터/어댑터 SHA-256·패키지 버전)를 기록합니다.

서빙·공정성. 튜닝 모델과 미튜닝 기준 모델은 모두 같은 bf16 서버와 greedy 디코딩으로 평가합니다. 이전에는 기준 모델을 다른 스택의 4bit 양자화로 서빙했으나, 같은 조건에서 다시 평가해도 두 모델의 성능 차이는 유지됐습니다.

평가 분할. (a) **고정 분할:** 홀드아웃 6사례 $\times$ 10인스턴스(60건) + seen-정책/unseen-인스턴스 56건. (b) *Leave-case-out(LOCO)*: 같은 레시피로 6-fold 재훈련해 **34개 정책 전부가 정확히 한 번씩 unseen**(340건, 클러스터 34개).

5. 결과

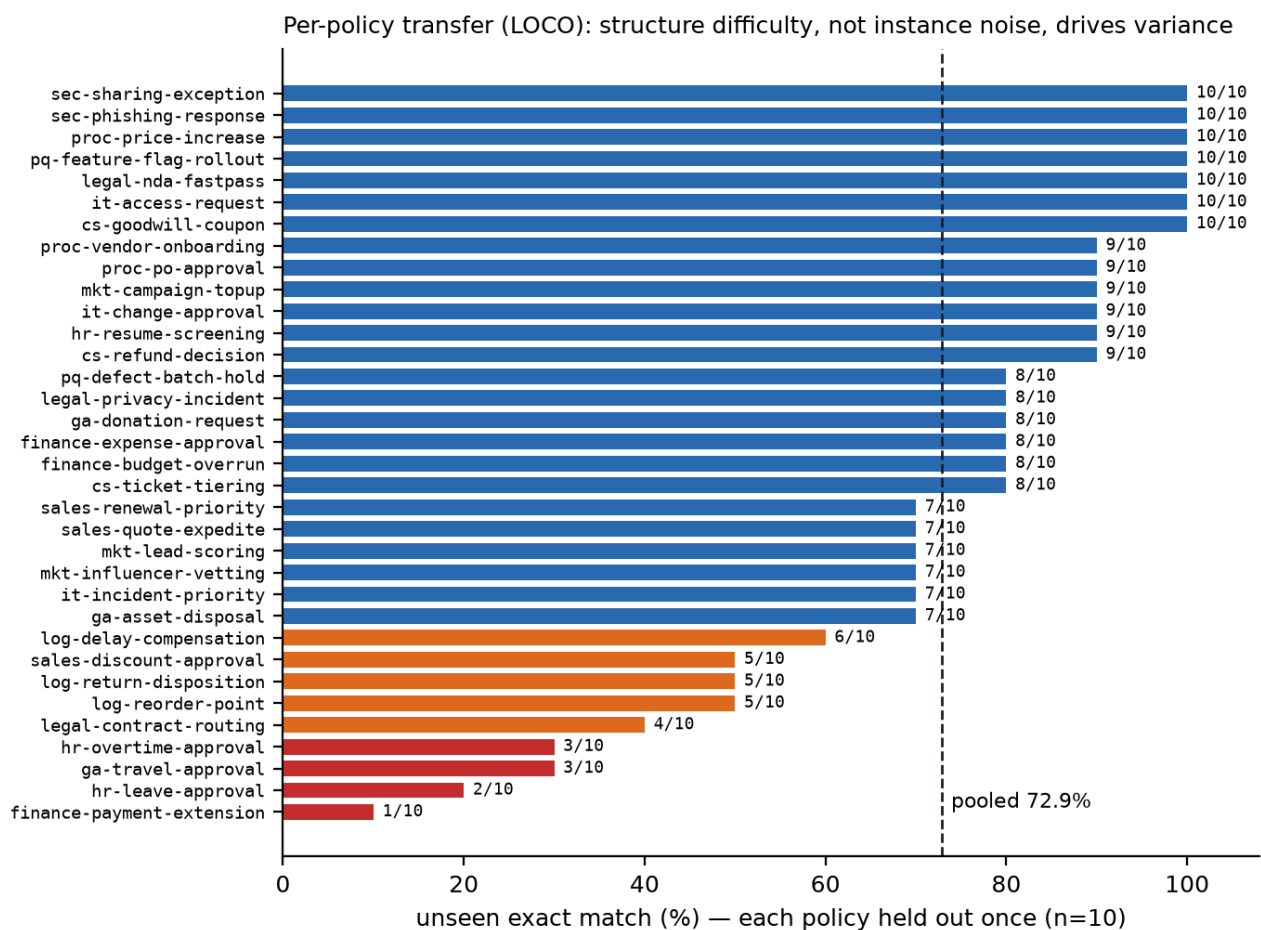
5.1 처음 보는 정책으로의 전이



모델 (동일 서빙)	unseen 정확	95% CI (클러스터)	verdict	완화
raw 3B, 제로샷	11.7%	[0.0, 25.0]	50.0%	11.7%

모델 (동일 서빙)	unseen 정확	95% CI (클러스터)	verdict	완화
raw 7B, 제로샷	15.0%	[3.3, 26.7]	53.3%	15.0%
raw 7B, 3-shot ICL (타 사례의 풀이 예시)	50.0%	[28.3, 71.7]	75.0%	51.7%
tuned 7B (고정 분할)	68.3%	[53.3, 83.3]	86.7%	71.7%
tuned 7B (LOCO, 34개 정책 전부 unseen 1회)	72.9%	[64.7, 80.9]	87.4%	73.8%

정책 단위 교차 검증에서도 성능 향상은 특정 분할의 우연으로 보이지 않았습니다. 여러 훈련 시드에서 실행 간 차이는 있었지만, 어느 시드에서도 미튜닝 기준 모델 수준으로 내려가지는 않았습니다. 인컨텍스트 예시만으로도 성능은 개선되지만, 파인튜닝은 그 위에 추가적인 향상을 보이며 매 요청에 예시를 넣는 비용도 없앱니다. 정책별 결과의 차이는 인스턴스 노이즈보다 정책 구조의 난이도에서 나왔습니다. 특히 날짜와 한도 계산이 섞인 승인 정책이 어려웠으며, 이는 §5.3에서 확인하는 계산의 경계와 맞닿아 있습니다.



5.2 무엇을 배웠나: 반사실 통제

모델이 정책을 읽은 것인지, 단지 세상의 상식이나 원래 정책을 따른 것인지 구분하기 위해 반사실 통제를 수행했습니다. 홀드아웃 사례의 모든 규칙과 fallback에서 approve와 reject를 뒤집고, 원래 정책과 다른 결론이 나오는 레코드만 평가했습니다. 이 데이터에서는 상식이나 원래 정책을 따르면 구조적으로 정답을 맞힐 수 없습니다.

모델	반사실 정확 (n=50)
tuned 7B	88.0%
raw 7B	48.0%

튜닝 모델은 뒤집힌 정책에 맞춰 결론을 바꿉니다. §5.1의 결과와 함께 보면, 파인튜닝이 가르친 것은 특정 정책의 내용이나 세상의 사전지식이 아니라 **컨텍스트에 주어진 정책을 적용하는 방법**이라는 근거가 됩니다. (approve/reject 판정이 없는 한 사례는 뒤집을 수 없어 제외했습니다. 이 통제의 정확도가 일반적인 새 정책 평가보다 높은 것은, 평가 레코드가 명시적 판정 규칙을 지나므로 인용할 근거가 더 분명하기 때문입니다.)

5.3 음성 통제: 같은 레시피는 계산을 못 배운다 — 프롬프팅도 마찬가지다

동일 레시피(같은 트레이너·게이트·코퍼스 규모 300예시)를 같은 제품군의 *파생* 과제에 적용했습니다: 고객의 갱신 가격 파일(22개 수치 필드: 좌석 올림, 성장률, 밴드 조희, 할인 상한, 월·연 총액)을 홀드아웃 고객에 대해 바이트 정확도로 재생성, 결정론 게이트가 채점.

파일	게이트 (exact)	수치 필드 정확도	실패 지점
SFT 7B (일반 타깃)	0/6	—	계산 값 오답
SFT 7B (CoT 타깃)	0/6	86.4% (114/132)	나눗셈 2필드; 1건은 밴드 연쇄 오류
프런티어, 같은 프롬프트, 도구 없음	0/6	93.6% (103/110)	거의 오로지 나눗셈 2필드

이 결과의 목적은 산술 실패 자체를 새롭게 발견하는 데 있지 않습니다. 산술을 도구로 처리하는 것은 이미 널리 쓰이는 방법입니다. 여기서 음성 통제는 “더 많이 학습하면 계산도 해결된다”는 해석을 배제합니다. 계산 과정을 포함한 학습 목표를 써도, 더 큰 프런티어 모델을 도구 없이 사용해도 같은 정확성 게이트를 통과하지 못했습니다. 반면 필드별 결과를 보면 모델은 대부분의 규칙과 값을 맞히며, 실패는 계산이 필요한 지점에 집중됩니다. 따라서 계산은 실행마다 모델이 도구를 선택하도록 두기보다 코드로 한 번 컴파일하는 편이 낫습니다. 반복 실행에서는 LLM이 루프에 남지 않기 때문입니다. 이로부터 도출되는 배치 원칙은 명확합니다. 파생 계산은 코드 계층에, 정책 판단은 결정론 게이트를 통과한 훈련된 소형 모델 계층에, 선언된 재량 범위는 추천과 사람 승인에 맡깁니다.

5.4 배치 경제학: 손익분기는 두 번째 실행

컴파일의 경제성은 한 번의 준비 비용을 반복 실행에서 회수할 수 있는지에 달려 있습니다. 갱신 파이프라인을 측정한 결과, 기록된 에이전트 세션은 실행마다 프런티어 토큰을 사용하지만 컴파일과 소형 모델 승격 뒤의 실행은 프런티어 토큰 없이 같은 출력을 재현했습니다. 컴파일의 입력은 조직이 이미 한 번 수행한 작업 세션이고, 컴파일 자체도 결정론적인 로컬 연산입니다. 따라서 이 방식은 이른 반복 단계부터 토큰 비용을 줄일 수 있습니다. 새로운 파라미터 값이 필요한 경우에만 프런티어 모델로 에스컬레이션하며, 그 결과는 상류 출력 지문과 함께 캐시해 이후 실행의 비용을 없앱니다. 물론 온톨로지와 정책을 선언하는 사람의 비용은 남습니다. 그러므로 이 파이프라인은 반복되는 결정일수록 유리하며, 이미 수행한 세션에서 시작하는 record-first 설계가 초기 준비 비용을 낮춥니다.

6. 논의와 한계

왜 정책 자체를 가중치에 넣지 않는가? 가중치에 내재화한 정책은 빠르게 답할 수 있지만, 정책이 바뀌는 순간 낡고 어떤 규칙을 적용했는지도 설명하기 어렵습니다. 이 접근에서는 정책 파일이 배포 산물이며, 재훈련은 정책 변경이 아니라 정책을 적용하는 *기술*이 약해질 때만 필요합니다.

정책 결정을 넘어. 이 방식은 승인 워크플로에만 묶이지 않습니다. 학습 데이터는 문서, 사례 레코드, 근거를 갖춘 구조화된 답변으로 이루어지며, 반사실 통제는 의미를 프로그램으로 뒤집을 수 있는 다른 문서에도 적용할 수 있습니다. 절차는 컨텍스트에 두고 실행 기술을 학습시키는 런북, 기준은 컨텍스트에 두고 채점 기술을 학습시키는 루브릭이 가까운 다음 대상입니다. 다만 각 영역에는 결과와 근거를 검사할 결정론적 채점기가 필요합니다. 이것이 적용 범위를 넓히는 실질적인 병목입니다. 이 논문은 기록된 에이전트 세션을 코드·규칙·캐시 계층으로 컴파일하는 더 큰 시스템에서, 검증을 거친 소형 모델이 판단을 맡는 한 계층을 다룹니다.

문서 클래스	필요한 결정론 채점기	현황
선언적 결정 정책	규칙 엔진 (존재)	측정 완료 — 본 논문
에스컬레이션 매트릭스	표 조회 검사기	근접 — 카탈로그 일부 사례가 이 형태
컴플라이언스 체크리스트	항목별 규칙 검사기	미착수
채점 루브릭	정답지 채점기	미착수
지원 런북	상태머신 검사기	미착수

산문 정책으로 가는 길. 이 실험에서는 규칙 필드명과 레코드 키가 일치하므로 엔티티 정합 문제가 없습니다. 다음 단계에서는 정책 적용 모델은 고정된 채, 앞단에 산문을 선언 규칙으로 바꾸는 정책 컴파일 단계를 두어야 합니다. 그때도 성능이 유지되는지와 반사실 통제가 계속 성립하는지를 측정해야 합니다.

순환성. 라벨과 채점이 같은 엔진에서 나온다는 한계가 있습니다. 근거 성립 검사, 완화 지표, 반사실 통제, 실행 환경 고정으로 그 위험을 줄였지만, 주석자 간 일치도를 갖춘 사람 라벨 평가는 향후 과제로 남습니다.

규모. 실험은 34개 정책, 하나의 7B 기반 모델, 한국어 프롬프트와 혼합 언어 필드라는 제한된 환경에서 수행했습니다. 따라서 이 결과는 새로운 정책에 정책 적용 기술이 전이될 가능성을 보인 증거이지, 모든 모델·언어·정책에 대한 일반적 보장은 아닙니다.

7. 재현성

코퍼스 생성기, 트레이너(핀 고정·런별 매니페스트), 평가 코드, 모든 원자료와 이 초고가 한 저장소에 있습니다: `examples/org/`(카탈로그·엔진·코퍼스), `tools/remote-train/`(트레이너·LOCO·통제 러너), `examples/org/decision-slm/`(RESULTS.md·eval_history.json·loco/results.json·c3c_controls.json), `examples/demo/customer-renewal-bench/slm-training/`(산술 팔·controls.json). 이 논문의 모든 표는 시드 고정 생성기와 저장된 모델 출력에서 재생 가능합니다.

제출 전 TODO (영어판과 공유)

- ☐ §2 관련 연구 산문화.
- ☐ refs.bib 저자 완성(제목·ID는 초록 검증 완료).
- ☐ 한국어판은 영어판 개정 시 함께 갱신(정본은 영어판).